MODULE 14 · DOMAIN 3 - DATA LOADING, UNLOADING, AND CONNECTIVITY

COPY INTO, error handling and unloading

Step two of the load path, and the same command run backwards.

DOMAIN 3 of 5 WEIGHT 18% MODULE 14 of 31

admin@sthithapragna.com

COPY is **idempotent by default**. Run it twice and you do not get the data twice.

Load metadata, and the 64-day boundary

- Snowflake records which files have been loaded into each table. **This load metadata expires after 64 days.**
- Within 64 days, COPY knows a file's status and **skips it** rather than duplicating the data.
- After 64 days the status is **unknown**, and COPY **skips the file by default** to avoid an accidental reload.
- **LOAD_UNCERTAIN_FILES = TRUE** loads files with expired metadata, while still using metadata where it exists.
- **FORCE = TRUE** ignores load metadata entirely and loads everything. Default is FALSE.

> ON THE EXAM
>
> Note the asymmetry: inside 64 days a file is skipped because it WAS loaded. Outside 64 days it is skipped because Snowflake cannot tell.

PURGE, and where Snowpipe differs

- **PURGE = TRUE** removes files from the stage after a successful load, which keeps storage costs down.
- **If the purge fails, no error is returned.** Confirm with LIST rather than assuming.
- **Snowpipe does not support PURGE.** Use REMOVE periodically, or cloud storage lifecycle rules.
- Snowpipe's load metadata lives on the **pipe object, not the table**, and is retained **14 days**, not 64.

> ON THE EXAM
>
> Recreating a pipe wipes its load history. Files already loaded can then be reloaded, which is a real duplication risk.

ON_ERROR, and two different defaults

VALUE	WHAT IT DOES
ABORT_STATEMENT	Stop the entire load if any error is found. The default for bulk COPY.
CONTINUE	Load every row that can be loaded, and skip the bad ones.
SKIP_FILE	Skip the whole file if any error is found. The default for Snowpipe.
SKIP_FILE_n	Skip the file once n error rows have been found.
SKIP_FILE_n%	Skip the file once the proportion of error rows reaches n percent.

ON THE EXAM

Bulk COPY defaults to ABORT_STATEMENT. Snowpipe defaults to SKIP_FILE. Same option, different default, and questions exploit that.

Validating before you load

- **VALIDATION_MODE** checks the files and returns results. **No data is loaded** when it is specified.
- **RETURN_n_ROWS** returns n rows if no errors are found. A quick sanity check on parsing.
- **RETURN_ERRORS** returns errors across the files. **RETURN_ALL_ERRORS** includes errors from previously failed loads.
- The **VALIDATE** table function inspects a load that has *already run*, rather than a hypothetical one.
- **XML is not supported** with VALIDATION_MODE.

> ON THE EXAM
>
> VALIDATION_MODE looks forward at files you have not loaded. VALIDATE looks backward at a COPY you already ran.

Transforming during load

SUPPORTED

Simple, row-by-row work

- Reorder columns
- Omit columns you do not want
- Cast data types
- Simple SQL expressions on staged columns

NOT SUPPORTED

Anything needing other rows

- Joins to other tables
- Aggregations and GROUP BY
- Filters and WHERE clauses
- FLATTEN

THE TIEBREAKER

The rule underneath: a COPY transformation can only look at the row in front of it. Anything needing another row is out.

Matching columns by name

- By default, staged columns map to target columns **by position**. The first column goes to the first column.
- **MATCH_BY_COLUMN_NAME** maps by name instead: **CASE_SENSITIVE**, **CASE_INSENSITIVE**, or **NONE**.
- That makes loads resilient when a source system reorders its columns or adds new ones.
- **INCLUDE_METADATA** maps METADATA$ columns, such as source filename and row number, into target columns.

> ON THE EXAM
>
> Positional matching fails silently when a source adds a column. Everything shifts one place and the load still succeeds.

Unloading is the same command backwards, and it is **less** restricted than loading.

Unloading options worth knowing

OPTION	WHAT IT DOES
SINGLE	Write one file instead of many. Default is FALSE.
MAX_FILE_SIZE	Maximum size of each output file. Default is 16 MB.
PARTITION BY	An expression splitting rows into separate files, producing a directory structure.
HEADER	Include column names as the first row. Default is FALSE.
OVERWRITE	Replace files already at the target location.

ON THE EXAM

The 16 MB default surprises people. Unload a large table without raising MAX_FILE_SIZE and you get a great many small files.

How this module gets tested

THE QUESTION SHAPE	THE ANSWER
'We re-ran COPY and nothing loaded'	Load metadata. Use FORCE to reload deliberately.
'One bad row killed a 5,000-file load'	ABORT_STATEMENT is the bulk default. Use CONTINUE or SKIP_FILE.
'Check the files without loading them'	VALIDATION_MODE. No data is loaded.
'Join to a lookup table during load'	Not supported. Load first, transform after.
'The source added a column and data went wrong'	Positional matching. Use MATCH_BY_COLUMN_NAME.
'The partner needs one CSV with headers'	SINGLE = TRUE, HEADER = TRUE, and raise MAX_FILE_SIZE.

ON THE EXAM

Six shapes. The first two are the ones that appear most often, and both are about a default nobody chose.

A pipeline loads daily files from an external stage. A bug is found in a downstream transformation. The team corrects the target table, then re-runs the original COPY statement to reload the last week of source files.

The statement completes successfully, reports zero rows loaded, and no error appears. The files are still present in the stage and were staged four days ago.

What is happening, and how should they reload?

A The files exceed the 64-day metadata window. Set LOAD_UNCERTAIN_FILES = TRUE.

B Load metadata records these files as already loaded. Use FORCE = TRUE to reload them.

C PURGE removed the files after the first load, so there is nothing left to read.

D The COPY was aborted by ON_ERROR. Set ON_ERROR = CONTINUE and re-run.

◆ PAUSE AND DECIDE

What is happening, and how should they reload?

A Exceeds the 64-day window. LOAD_UNCERTAIN_FILES.

The files were staged four days ago, so their status is known rather than uncertain. Right option, wrong situation.

B Already loaded. Use FORCE = TRUE.

COPY is idempotent by default. Within 64 days it knows these files were loaded and skips them. FORCE ignores that metadata.

C PURGE removed the files.

The scenario states the files are still present in the stage. It also would have produced an error about missing files, not zero rows.

D Aborted by ON_ERROR.

An abort produces an error, not a clean success. The statement completed successfully with zero rows.

THE SIGNAL

'Completes successfully, zero rows, no error' is the signature of load metadata. A failure would look quite different.

> A nightly bulk load reads about 5,000 files. One night it fails after roughly forty minutes with an error about an unparseable date in a single row. Nothing is loaded at all.
>
> The team wants the load to complete, and to be told afterwards which rows could not be parsed. Losing the whole night's data to one bad record is unacceptable.

What should they change?

A. Nothing in COPY. Add a Snowpipe pipe, which skips bad files automatically.

B. Set VALIDATION_MODE = RETURN_ALL_ERRORS on the nightly load.

C. Set FORCE = TRUE so the load ignores the failed rows and continues.

D. The default is ABORT_STATEMENT. Set ON_ERROR = CONTINUE and review COPY_HISTORY afterwards.

◆ PAUSE AND DECIDE

What should they change?

A Add a Snowpipe pipe.

Snowpipe does default to SKIP_FILE, and redesigning the ingestion architecture to change an error-handling default is disproportionate.

B VALIDATION_MODE = RETURN_ALL_ERRORS.

Validation mode loads no data at all. The nightly load would report errors and never populate the table.

C FORCE = TRUE.

FORCE governs load metadata and reloading. It has nothing to do with error handling.

D ON_ERROR = CONTINUE, then review COPY_HISTORY.

Loads every row that parses, skips the rest, and the rejected rows are visible afterwards. Exactly the stated requirement.

THE SIGNAL

ABORT_STATEMENT is the bulk default nobody chooses. Note that the failed run will not appear in COPY_HISTORY, because nothing was ingested.

> An engineer wants a single COPY statement to load staged CSV files into a target table while looking up a region code from an existing dimension table, and while summing several line items into a total column.
>
> They have written a SELECT from the stage with a join and a GROUP BY, and it fails.

Why does it fail, and what is the right pattern?

A. The SELECT must reference columns as $1, $2. Rewriting it positionally will allow the join.

B. MATCH_BY_COLUMN_NAME must be set before a query can be used as the source of a COPY.

C. COPY transformations do not support joins or aggregations. Load the raw data first, then transform.

D. Joins are supported, but only against tables in the same schema as the stage.

◆ PAUSE AND DECIDE

Why does it fail, and what is the right pattern?

A Rewrite the SELECT positionally.

Positional references are genuinely required, and they do not unlock joins. The restriction is on the operation, not the syntax.

B Set MATCH_BY_COLUMN_NAME first.

That controls how staged columns map to target columns. It is unrelated to whether a join is permitted.

C No joins or aggregations. Load raw, then transform.

Documented. A COPY transformation sees one row at a time, so anything needing another row or table is out.

D Joins work within the same schema.

Invented qualification. Joins are not supported in a COPY transformation at any scope.

THE SIGNAL

Two separate violations here: a join and an aggregation. Either alone would fail. Both need the load-then-transform pattern.

A partner requires a single CSV file containing about 900 MB of data, with a header row, delivered to an external stage each week. The partner's loader cannot handle multiple files or headerless data.

An engineer runs COPY INTO with a file format of type CSV and finds dozens of small files in the stage, none with headers.

Which options are needed?

A. SINGLE = TRUE, HEADER = TRUE, and MAX_FILE_SIZE raised above the 16 MB default.

B. PARTITION BY on a constant expression, which forces all rows into one partition and one file.

C. OVERWRITE = TRUE, which replaces the many files with one combined file.

D. Nothing. Unload to an internal stage instead, which always produces a single file with headers.

◆ PAUSE AND DECIDE

Which options are needed?

A SINGLE, HEADER, and a raised MAX_FILE_SIZE.

Three defaults, three symptoms. SINGLE is FALSE, HEADER is FALSE, and MAX_FILE_SIZE is 16 MB, which 900 MB comfortably exceeds.

B PARTITION BY a constant.

Partitioning splits output into more files, not fewer. It also cannot omit the partition column from the files.

C OVERWRITE = TRUE.

Controls whether existing files at the target are replaced. It does not combine output into one file.

D Unload to an internal stage.

Stage type does not change file count or headers. The same defaults apply wherever you unload to.

THE SIGNAL

Every symptom traces to a default: SINGLE FALSE, HEADER FALSE, MAX_FILE_SIZE 16 MB. None of them is what a partner usually wants.

What to carry into the next module

- **Load metadata expires after 64 days.** FORCE reloads deliberately; LOAD_UNCERTAIN_FILES handles expired status.
- **ABORT_STATEMENT is the bulk default. SKIP_FILE is the Snowpipe default.** Same option, different defaults.
- **VALIDATION_MODE loads nothing.** It looks forward at files; VALIDATE looks backward at a COPY already run.
- **No joins, aggregations, filters or FLATTEN during load.** Unloading has no such limit, because the source can be any SELECT.
- **Unload defaults:** SINGLE FALSE, HEADER FALSE, MAX_FILE_SIZE 16 MB.

> ON THE EXAM
>
> Next: continuous ingestion. Snowpipe, Snowpipe Streaming, streams, tasks and dynamic tables. The densest module in the course.